

2026 Introduction to VLSI and System-on -Chip Design

Final Lab Matrix Multiplication

Objection

In this lab, students are required to implement a hardware design for **three 2x2 matrix multiplication** and learn RTL simulation and synthesis.

Lab Download

1. Download the provided environment using the following command.
`tar xvf /home/TA00/VLSI_UG_2026/Lab_final.tar`
2. The extracted LAB directory contains:
 - **00_TESTBED**: Contains the testbench and pattern files .
 - **01_RTL**: Your workspace for the Verilog design (**MATRIX_MULT.v**).
 - **02_SYN**: Synthesis scripts and working directory.
 - **03_GATE**: Gate-level netlist and simulation files.

Design Description

The MATRIX_MULT module performs the multiplication of three 2x2 matrices, Matrix A, Matrix B and Matrix C. Both matrices contain 4-bit unsigned integers.

$$\begin{bmatrix} a_1 & a_2 \\ a_3 & a_4 \end{bmatrix} \times \begin{bmatrix} b_1 & b_2 \\ b_3 & b_4 \end{bmatrix} \times \begin{bmatrix} 6 & 5 \\ 7 & 2 \end{bmatrix} = \begin{bmatrix} d_1 & d_2 \\ d_3 & d_4 \end{bmatrix}$$

Mat. A **Mat. B** **Mat. C**
(Hardcoded) **Mat. D**

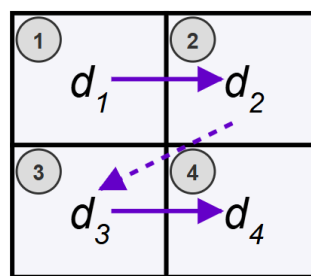
Fig. 1. 3x3 Matrix Multiplication Example

1. **Input Phase (8 Cycles)**: When the **in_valid** signal is high, the design receives one element of a matrix per clock cycle, and then the input port (**in_data**) is 4 bits wide. The testbench will transmit Matrix A completely, followed immediately by Matrix B. **Therefore, the continuous input phase**

lasts for exactly 8 cycles (Cycles 1 to 4 for Matrix A's elements, and Cycles 5 to 8 for Matrix B's elements).

2. **Calculation Phase:** The core operation is a two-stage matrix multiplication to compute Matrix $D = A \times B \times C$, where all matrices are of size 2×2 . In the first stage, an intermediate matrix is computed by multiplying the elements of the i -th row of Matrix A with the corresponding elements of the j -th column of Matrix B, summing the 2 resulting products. Since multiplying two 4-bit numbers results in an 8-bit product, and summing 2 products adds 1 bit of growth, each intermediate element requires 9 bits. In the second stage, the final elements of Matrix D are computed by multiplying the i -th row of this intermediate matrix with the j -th column of Matrix C, summing another 2 resulting products. Since the elements of Matrix C require up to 3 bits, multiplying a 9-bit intermediate value with a 3-bit constant results in a 12-bit product. Summing 2 of these products requires an additional bit; thus, to ensure precision and prevent overflow, **each final output element in Matrix D is allocated 13 bits.**
3. **Output Phase (4 Cycles):** Once the computation is ready, the module must assert the `out_valid` signal and begin outputting the resulting 2×2 Matrix D. The output is provided one element at a time (13 bits per cycle) through the `out_data` port. **The output sequence must strictly follow row-major order: starting from Row 0 (elements 0 to 1), followed by Row 1 (elements 0 to 1). This continuous output phase lasts for exactly 4 cycles.**

Mat. D Output (1 Element per Cycle)



Grey circles indicate the output cycle number.

Fig. 2. Output order in 4 consecutive cycles

Signal Description

Signal	In / Out	Bit Width	Description
clk	In	1	System Clock.
rst_n	In	1	Asynchronous active-low reset.
in_valid	In	1	High when the input <code>in_data</code> is valid.
in_data	In	4	A single 4-bit element of the input matrices.
out_valid	Out	1	High when the output <code>out_data</code> is valid.
out_data	Out	13	A single 13-bit element of the calculated output matrix.

Table. 1. I/O port list

Design Constraints

01_RTL

1. Top module name: `MATRIX_MULT.v`
2. The reset signal (`rst_n`) would be given only once at the beginning of simulation. All output signals should be reset after the reset signal is asserted.
3. The `out_data` should be zero after your `out_valid` is pulled down.
4. The `out_valid` signal cannot overlap with the `in_valid` signal at any time.
5. The `out_valid` signal should be high in **4 consecutive cycles**.
6. The next inputs will come in the next 3 ~ 7 negative edges of the clock after your `out_valid` is pulled down.
7. In this lab, you must implement the design using DesignWare IPs. The use of regular arithmetic operators, specifically '*' and '+', is strictly prohibited anywhere in the design, including all comments.

02_SYN

1. The synthesis result cannot contain any **latch**.
2. The input / output delay is set to 0.5 * (clock period), and the output loading is set to 0.05.
3. You can adjust your clock period by yourself. The precision of the clock period is 0.1, for example, 45.0 is allowed, 44.55 is not allowed.

- After synthesis, the design must meet timing requirements. The area report is valid only if the final slack reported in `MATRIX_MULT.timing` is **MET**.

```

Startpoint: in_valid (input port clocked by clk)
Endpoint: mat_a_reg_1_0_0_0_
(rising edge-triggered flip-flop clocked by clk)
Path Group: clk
Path Type: max

```

Point	Incr	Path
clock clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
input external delay	10.00	10.00 f
in_valid (in)	0.00	10.00 f
U1789/Y (INVX1)	0.26	10.26 r
U1795/Y (NOR2XL)	0.12	10.38 f
U3208/Y (NAND3BX1)	0.15	10.53 r
U1791/Y (INVXL)	0.11	10.64 f
U3339/Y (NAND3XL)	0.14	10.78 r
U3602/Y (OR2X1)	0.80	11.58 r
U1799/Y (CLKINVX3)	0.62	12.20 f
U3659/Y (A022X1)	0.38	12.59 f
mat_a_reg_1_0_0_0_/D (DFFQX1)	0.00	12.59 f
data arrival time		12.59
clock clk (rise edge)	20.00	20.00
clock network delay (ideal)	0.00	20.00
clock uncertainty	-0.10	19.90
mat_a_reg_1_0_0_0_/CK (DFFQX1)	0.00	19.90 r
library setup time	-0.23	19.67
data required time		19.67
data arrival time		-12.59
slack (MET)		7.08

Fig. 3. Timing report with no timing violation

- You can use `08_check` to check your synthesis result.

```

[chfan@localhost 02_SYN]$ ./08_check
=====
--> V Latch Checked!
--> V Width Mismatch Checked!
--> V No Error in syn.log!
--> V Timing (MET) Checked!
=====
--> V 02_SYN Success !!
=====
Cycle: 20.00
Area: 63126.305690
Gate count: 6326
Dynamic: Total Dynamic Power    = 923.2034 uW  (100%)
Leakage: Cell Leakage Power     = 53.2685 uW

[chfan@localhost 02_SYN]$ ./08_check
=====
--> X There is Latch in this design ---
--> V Width Mismatch Checked!
--> V No Error in syn.log!
--> V Timing (MET) Checked!
=====
--> X 02_SYN Fail !! Please check out syn.log file.
=====
Cycle: 20.00
Area: 63126.305690
Gate count: 6326
Dynamic: Total Dynamic Power    = 923.2034 uW  (100%)
Leakage: Cell Leakage Power     = 53.2685 uW

```

Fig. 4. `08_check` example

03_GATE

1. All requirements in 01_RTL must be met after including timing checks in Gate-Level simulation.

Waveform Examples

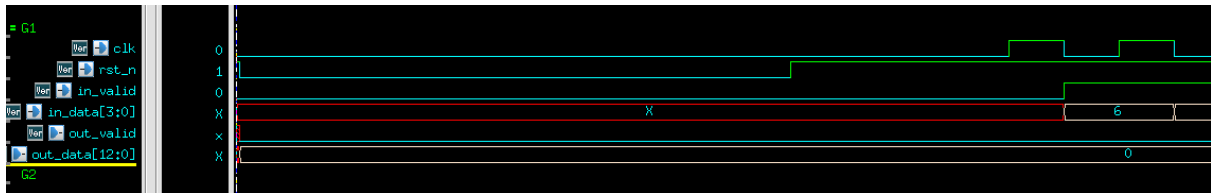


Fig. 5. The reset signal would be given only once at the beginning of simulation.

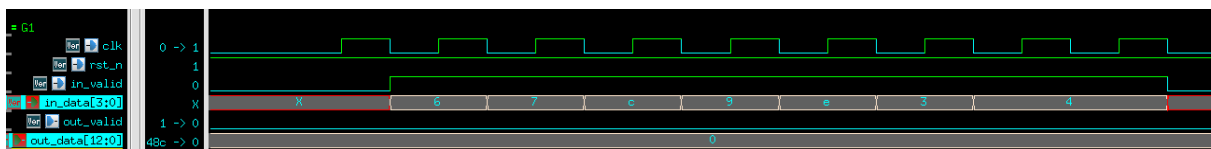


Fig. 6. input signal will be sent in 8 consecutive cycles.

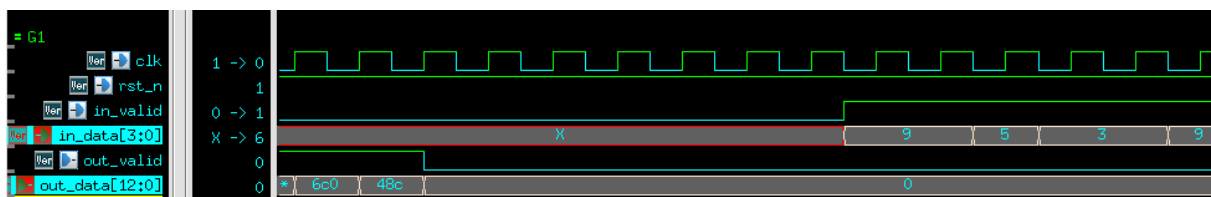


Fig. 7. The next input will come in the next 3 ~ 7 negative edges of the clock.

Grading Policy

1. Function Validity: 80%

Passing RTL and Gate-level simulation without timing violations.

2. Performance: 20%

Performance Score =

$$\text{Latency} * \text{Clock Cycle Time}^2 * \text{Area} * \text{Power}$$

※The **Latency** is the clock cycles between the falling edge of the in_valid and the rising edge of the first out_valid.

※**Clock Cycle Time** is defined by yourself.

Notes

- Reference Commands
 - 01_RTL/ (RTL simulation) ./01_run_vcs_rtl
 - 02_SYN/ (Synthesis) ./01_run_dc_shell
 - 03_GATE / (Gate-level simulation) ./01_run_vcs_gate

- How to change cycle time

You can adjust cycle time in 00_TESTBED/PATTERN.v and 02_SYN/syn.tcl.

※Note: Both cycle time configurations must be the same.

```
00_TESTBED > 00_PATTERN.v
1  ▾ `ifdef RTL
2    `define CYCLE_TIME 20.0
3  `endif
4  ▾ `ifdef GATE
5    `define CYCLE_TIME 20.0
6  `endif
```

Fig. 8. 00_TESTBED/PATTERN.v

```
02_SYN > 02_syn.tcl
1  #=====
2  #
3  # Synopsys Synthesis Scripts (Design Vision dcl mode)
4  #
5  #=====
6
7  #=====
8  # (A) Global Parameters
9  #=====
10 set DESIGN "MATRIX_MULT"
11 set CYCLE 20
12 set INPUT_DLY [expr 0.5*$CYCLE]
13 set OUTPUT_DLY [expr 0.5*$CYCLE]
```

Fig. 9. 02_SYN/syn.tcl